

人工智能导论

一、人工智能概述

形式化系统的性质：完备性、一致性、可判定性

Deepmind研制的AlphaGo算法没有使用哪个人工智能算法？逻辑推理

图灵奖获得者Judea Pearl将推理按照由易到难程度分成三个层次：关联、干预、反事实

以下哪种搜索方法属于启发式搜索？A*搜索

以下哪一种算法不属于监督学习算法？聚类

以下那种学习方法的学习目标是学习同一类数据的分布模式？无监督学习

人工智能是以机器为载体的所展现的人类智能，因此人工智能也被称为机器智能

从智能角度对人工智能进行分类：领域人工智能、通用人工智能、混合增强人工智能

人工智能的三种主流方法包括：符号主义、联结主义、行为主义

推理一般包括：归纳推理、演绎推理、因果推理

人工智能特点：至小有内、至大无外、多学科交叉内禀

按照学习算法对数据的利用方式不同，机器学习算法可以分为：监督学习、无监督学习、半监督学习

有信息搜索又称为启发式搜索，A*搜索/贪婪优先搜索是其中比较有代表性的算法

对抗搜索又称为博弈搜索，代表性算法包括：最大最小搜索、Alpha-Beta剪枝搜索、蒙特卡洛树搜索

二、逻辑与推理

命题逻辑

命题逻辑是应用一套形式化规则对以符号表示的描述性陈述进行推理的系统。

在命题逻辑中，一个或真或假的描述性陈述被称为原子命题，对原子命题的内部结构不做任何解析。

若干原子命题可以通过逻辑运算符来构成复合命题。

可以通过命题联结词对已有命题进行组合，得到新命题。

五种主要命题联结词： $\wedge$ （与）、 $\vee$ （或）、 $\neg$ （非）、 $\rightarrow$ （条件）、 $\leftrightarrow$ （双条件）

命题逻辑中的推理规则

名称	规则
假言推理	$\alpha \rightarrow \beta, \alpha \Rightarrow \beta$
与消解	$\alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_n \Rightarrow \alpha_i (i \leq i \leq n)$
与导入	$\alpha_1, \alpha_2, \cdots, \alpha_n \Rightarrow \alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_n$
双重否定	$\neg \neg \alpha \Rightarrow \alpha$
单项消解或单项归结	$\alpha \vee \beta, \neg \beta \Rightarrow \alpha$
消解或归结	$\alpha_1 \vee \alpha_2 \vee \cdots \vee \alpha_m, \neg \beta (\alpha_k = \beta) \Rightarrow \alpha_1 \vee \alpha_2 \vee \cdots \vee \alpha_{k-1} \vee \alpha_{k+1} \vee \cdots \vee \alpha_m$

命题范式

有限个简单合取式构成的析取式称为析取范式

有限个简单析取式构成的合取式称为合取范式

析取范式与合取范式统称为范式

一个析取范式是不成立的，当且仅当它的每个简单合取式都不成立

一个合取范式是成立的，当且仅当它的每个简单析取式都是成立的

任一命题公式都存在着与之等值的析取范式与合取范式

谓词逻辑

在谓词逻辑中，将原子命题进一步细化，分解出个体、谓词和量词，来表达个体与总体的内在联系和数量关系，这就是谓词逻辑研究内容。

全称量词： $\forall$ ，存在量词： $\exists$

全称量词与存在量词的转换（略）

谓词逻辑的推理规则

- 全称量词消去： $(\forall x)A(x) \Rightarrow A(y)$
- 全称量词引入： $A(y) \Rightarrow (\forall x)A(x)$
- 存在量词消去： $(\exists x)A(x) \Rightarrow A(c)$
- 存在量词引入： $A(c) \Rightarrow (\exists x)A(x)$

知识图谱推理

知识图谱可视为包含多种关系的图。在图中，每个节点是一个实体，任意两个节点之间的边表示这两个节点之间存在的关系。

一般而言，可将知识图谱中任意两个相连的节点及其连接边表示成一个三元组。

归纳学习

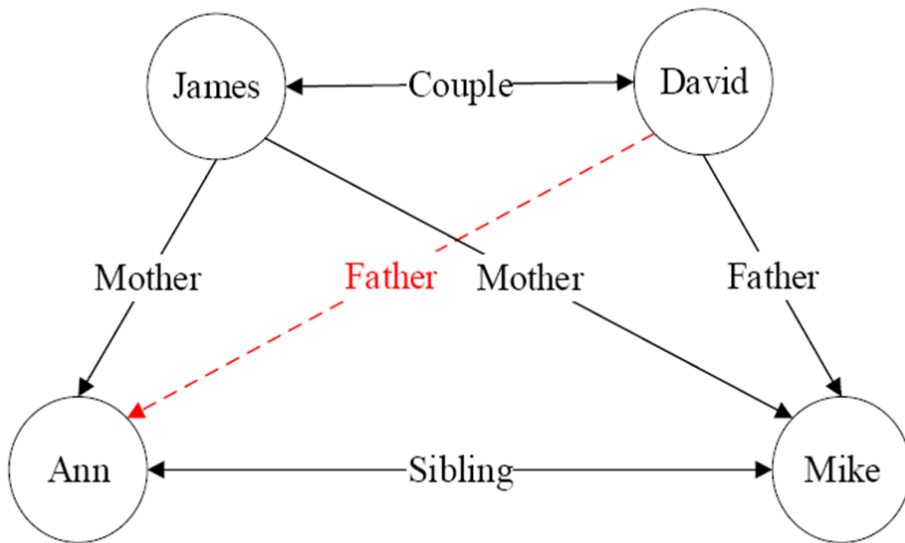
归纳逻辑程序设计（ILP）算法

ILP使用一阶谓词逻辑进行知识表示，通过修改和扩充逻辑表达式对现有知识归纳，完成推理任务。

作为ILP的代表方法，FOIL通过序贯覆盖实现规则推理。

FOIL(First Order Inductive Learner)

FOIL算法	
输入:	目标谓词 P , P 的训练样例（正例集合 E^+ 和反例集合 E^- ），其他背景知识
输出:	推导得到目标谓词 P 的推理规则
1	将目标谓词作为所学习推理规则的结论
2	将其他谓词逐一作为前提约束谓词加入推理规则，计算所得到推理规则的FOIL信息增益值，选取最优前提约束谓词以生成新推理规则，并将训练样例集合中与该推理规则不符的样例去掉
3	重复2过程，直到所得到的推理规则不覆盖任意反例



- 推理手段：正例+反例+背景知识 $\Rightarrow$ 假设
- 目标谓词： $Father(s, y)$
- 目标谓词只有一个正例 $Father(David, Mike)$
 - 反例一般在知识图谱中不会显式给出，但是可以构造出来。
- 只能在已知两个实体的关系且确定其关系与目标谓词相悖时，才能将这两个实体用于构建目标谓词的反例
- 背景知识：知识图谱中目标谓词以外的其他谓词实例化结果，如 $Sibling(Ann, Mike)$

• FOIL信息增益值计算方法如下：

$$FOIL_Gain = \widehat{m}_+ \cdot \left(\log_2 \frac{\widehat{m}_+}{\widehat{m}_+ + \widehat{m}_-} - \log_2 \frac{m_+}{m_+ + m_-} \right)$$

- 其中， $\widehat{m}_+$ 和 $\widehat{m}_-$ 是增加前提约束谓词后所得新推理规则覆盖的正例和反例的数量， m_+ 和 m_- 是原推理规则所覆盖的正例和反例数量

因果推理

辛普森悖论：总体样本上成立的某种关系在分组关系上正好相反

有向无环图

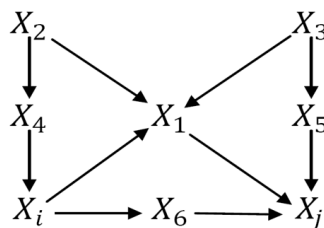
使用有向无环图描述变量的联合分布或者数据生成机制的模型，被称为贝叶斯网络。

联合分布概率由每个节点与其父节点之间条件概率的乘积给出：

$$P(x_1, x_2, \dots, x_d) = \prod_{j=1}^d P(x_j | x_{pa(j)})$$

因果推理：有向无环图（DAG）

- 一个有向无环图唯一地决定了一个联合分布
- 一个联合分布不能唯一地决定有向无环图
 - 反过来的结论不成立
 - 如联合分布 $P(X_1, X_2) = P(x_1)P(x_2|x_1) = P(x_2)P(x_1|x_2)$



有向无环图DAG

联合分布可表示为：

$$\begin{aligned} &P(X_1, X_2, X_3, X_4, X_5, X_6, X_i, X_j) \\ &= P(X_2) \times P(X_3) \times P(X_1 | X_2, X_3, X_i) \\ &\quad \times P(X_4 | X_2) \times P(X_5 | X_3) \times P(X_6 | X_i) \times P(X_i | X_4) \\ &\quad \times P(X_j | X_1, X_5, X_6) \end{aligned}$$

内生变量：有边指向的变量

外生变量：没有边指向的变量

D-分离

三种结构

- 链结构: $A \rightarrow C \rightarrow B$
- 分连结构: $A \leftarrow C \rightarrow B$
- 汇连结构: $A \rightarrow C \leftarrow B$

D-分离: 在DAG (有向无环图) 中, 如果节点A与B之间所有的路径 (不考虑方向) 在给定条件C下都是阻塞的, 那么A和B在条件C下是独立的。

这里的A, B, C既可以是单独的一个点, 也可以是点集。

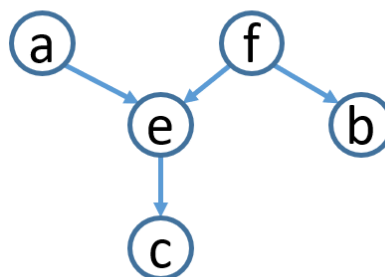
阻塞的两种情况: 路径中存在节点X

- X是链结构或分连结构节点, 且 $X \in C$
- X是汇连结构节点, 并且X或X的后代不包含在C中

例:

下面描述中正确的有

- ☐ A a和b是条件c下独立的
- ☐ B a和b是条件e下独立的
- ☒ C a和b是条件f下独立的



在该图中, 从a到b的路径只有a-e-f-b

- A: e是汇连结构点, e的后代是c, 包含在c中, 路径不是阻塞的, 故A错
- B: e是汇连结构点, e在e中, 路径不是阻塞的, 故B错
- C: f是分连结构点, $f \in f$, 路径是阻塞的, 故C对

D-分离方法可以用于判断因果图上任意变量之间的相关性和独立性。

在因果图上, 若节点X和节点Y之间的每一条路径都是阻塞的, 则称节点X和节点Y是有向分离的, 反之则称节点X和节点Y是有向连接的。

当两个节点是有向分离时, 意味着这两个节点相互独立。

干预和do算子

在DAG中, $do(X_i) = x'_i$ 表示将DAG中指向节点 X_i 的有向边全部切断, 并将 X_i 的值固定为常数 x'_i .在这样操作后,得到新的DAG中的变量联合分布为

$$P(x_1, x_2, \dots, x_d | do(X_i) = x'_i)$$

三、搜索求解

搜索算法基础

状态-动作-状态转移-代价-目标测试

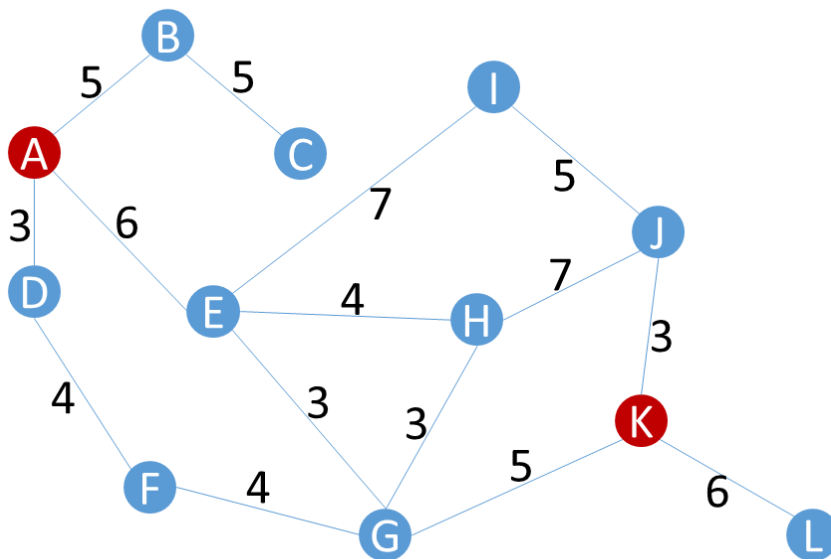
搜索树:搜索算法是一个构建搜索树的过程,从根结点(初始状态)开始,不断展开每个结点的后继结点,直到某个结点通过了目标测试.

搜索算法的评价指标:完备性,最优性,时间复杂度,空间复杂度.

剪枝搜索:有时候,主动放弃一些后继结点能够提高搜索效率而不会影响最终的搜索结果,甚至能解决无限循环的问题.

启发式搜索

以计算两城市之间的最短距离为例



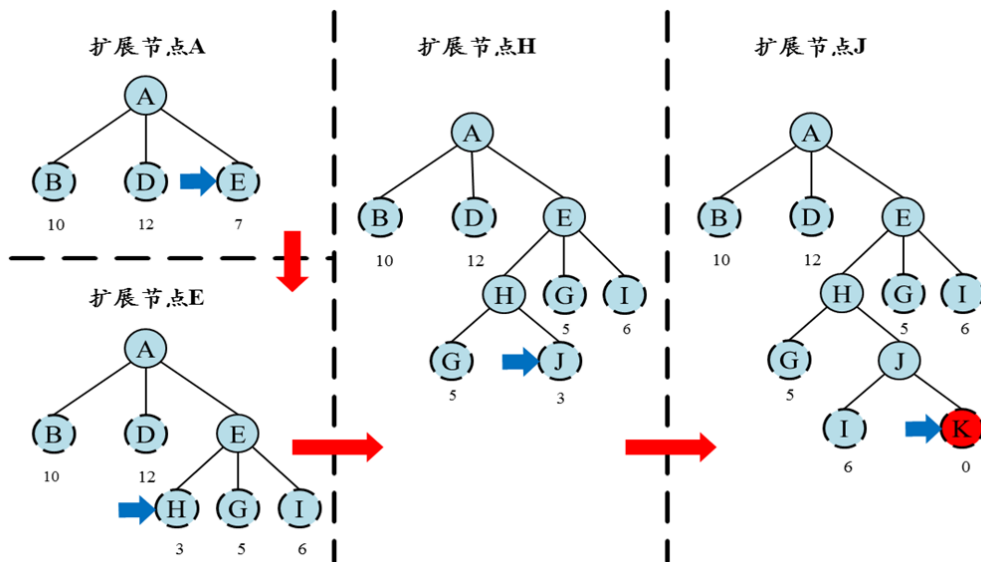
辅助信息:任一城市与终点城市K之间的最小值

状态	A	B	C	D	E	F	G	H	I	J	K	L
$h(n)$	13	10	6	12	7	8	5	3	6	3	0	6

- 评价函数 $f(n)$:从当前节点出发,根据评价函数来选择后续节点
- 启发函数 $h(n)$:从节点n到目标节点之间所形成路径的最小代价值,这里用两点之间的直线距离.

贪婪优先搜索

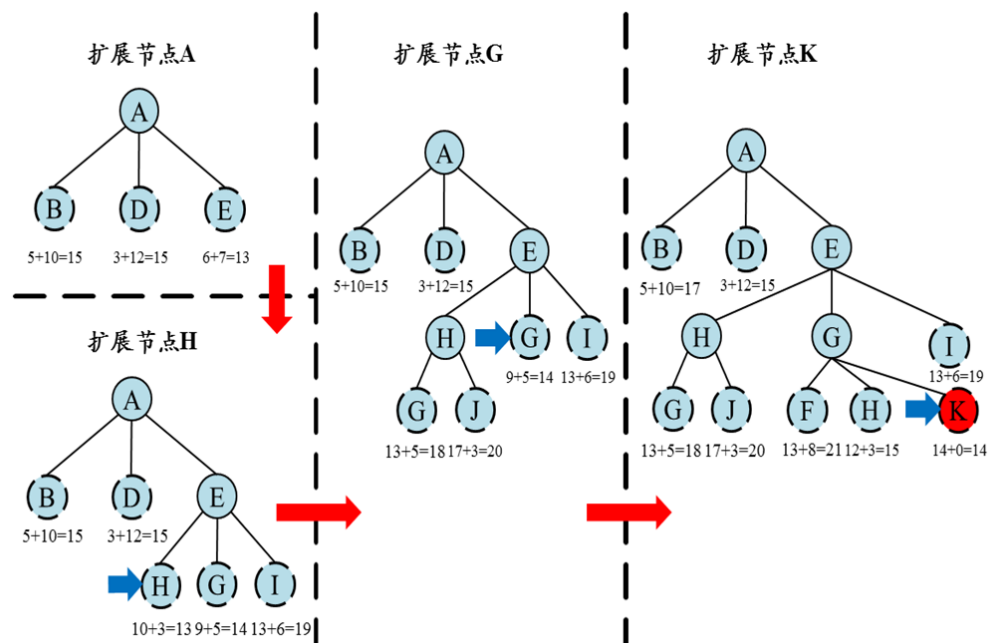
对贪婪优先搜索来说, $f(n) = h(n)$



A*算法

对A*算法来说, $f(n) = g(n) + h(n)$

这里的 $g(n)$ 代表从起始节点到节点 n 的开销代价值



A*算法分析

启发函数需要满足如下两种性质:

- **可容性:**对于任意节点 n ,有 $h(n) \leq h^*(n)$,如果 n 是目标节点,则有 $h(n) = 0$. $h^*(n)$ 是从节点 n 出发到达终点节点所付出的实际最小代价.可以这样理解满足可容性的启发函数,启发函数不会过高估计从节点 n 到终止节点所应该付出的代价.
- **一致性:**启发函数的一致性指满足条件 $h(n) \leq c(n, a, n') + h(n')$,这里 $c(n, a, n')$ 表示节点 n 通过动作 a 到达其相应的后继节点 n' 的代价.

满足一致性的启发函数一定满足可容性.

A*算法是完备的,如果存在解,A*算法肯定可以得到解,如果得不到解那么说明解一定不存在.

如果启发算法是可容的,那么树搜索的A*算法满足最优性.

如果A*算法采用的是图搜索策略,那么即使启发函数满足可容性条件,也不能保证算法的最优性.

使图搜索A*算法满足最优性的方法

- 方法一:当算法从不同路径扩展同一节点时,不保留最先探索的那条路径,而是**保留代价最小的那条路径**
- 方法二:要求启发函数满足**一致性**

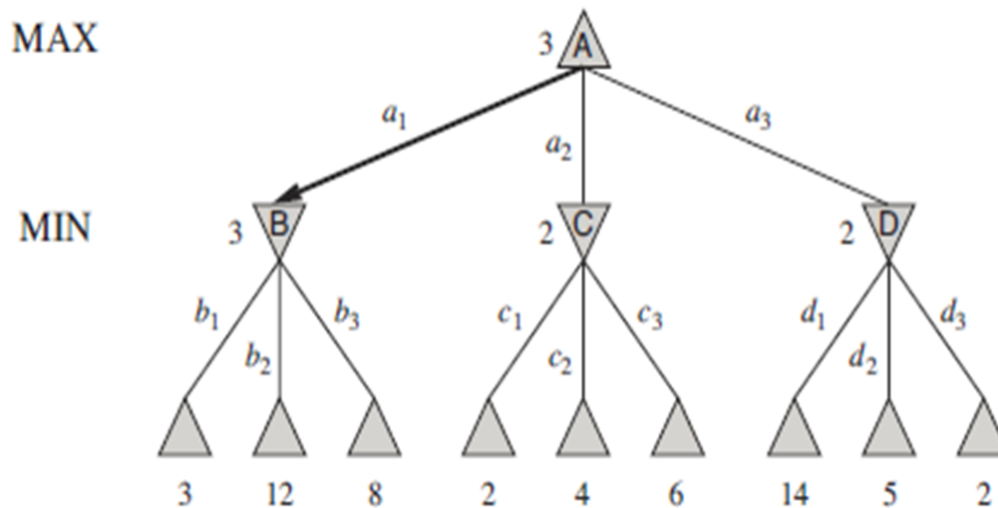
对抗搜索

对抗搜索也称为博弈搜索.在一个竞争的环境中,智能体之间通过竞争实现相反的利益,一方最大化这个利益,另一方最小化这个利益.

最小最大搜索

一层选最小值,一层选最大值.

下图中从叶节点往上看,min层选择下一层中的最小值,max层选择下一层(min层)中的最大值



优点:

- 算法是一种简单有效的对抗搜索手段
- 在对手也尽力而为的情况下,可以返回最优解

缺点:

- 如果搜索树极大,则无法在有效时间内返回结果

改善:

- 使用Alpha-Beta剪枝搜索来减少搜索节点
- 对节点进行采样,而非逐一搜索

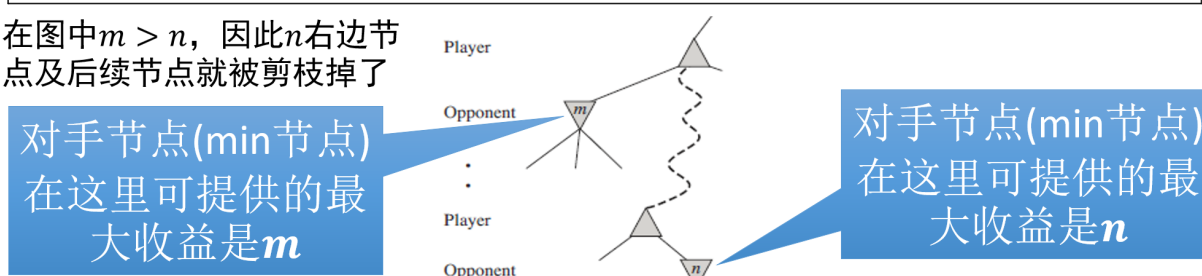
Alpha-Beta剪枝搜索

从minimax算法中剪去不影响最终结果的搜索分支

- Alpha值:MAX节点目前可以得到的最高收益
- Beta值:MIN节点目前可以给对手的最小收益

Alpha值(α)	玩家MAX(根节点)目前得到的最高收益
	假设 n 是MIN节点, 如果 n 的一个后续节点可提供的收益小于 α , 则 n 及其后续节点可被剪枝
Beta值(β)	玩家MIN目前给对手的最小收益
	假设 n 是MAX节点, 如果 n 的一个后续节点可获得收益大于 β , 则 n 及其后续节点可被剪枝
α 和 β 的值初始化分别设置为 $-\infty$ 和 ∞	

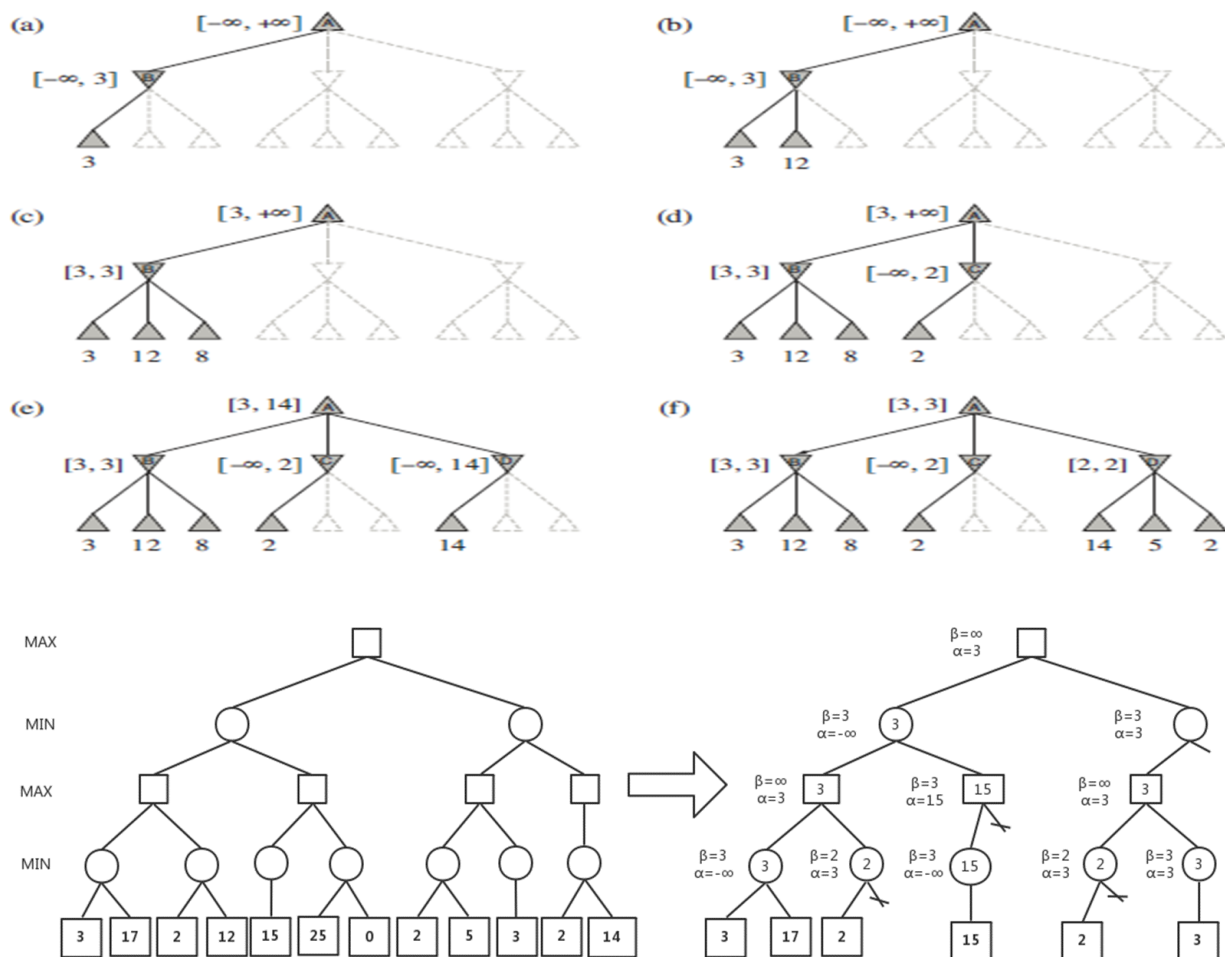
在图中 $m > n$, 因此 n 右边节点及后续节点就被剪枝掉了



- α 为可能解法的最大上界

- β 为可能解法的最小下界

- 如果节点 N 是可能解法路径中的一个节点, 则其产生的收益一定满足如下条件: $\alpha \leq \text{reward}(N) \leq \beta$ (其中 $\text{reward}(N)$ 是节点 N 产生的收益)
- 每个节点有两个值, 分别是 α 和 β 。节点的 α 和 β 值在搜索过程中不断变化。其中, α 从负无穷大($-\infty$)逐渐增加、 β 从正无穷大(∞)逐渐减少。如果一个节点中 $\alpha > \beta$, 则该节点的后续节点可剪枝。



如果节点次序恰到好处,Alpha-Beta剪枝的事件复杂度为 $O(b^{\frac{m}{2}})$,最小最大搜索的事件复杂度为 $O(b^m)$

蒙特卡洛树搜索

上限置信区间算法(UCB1):

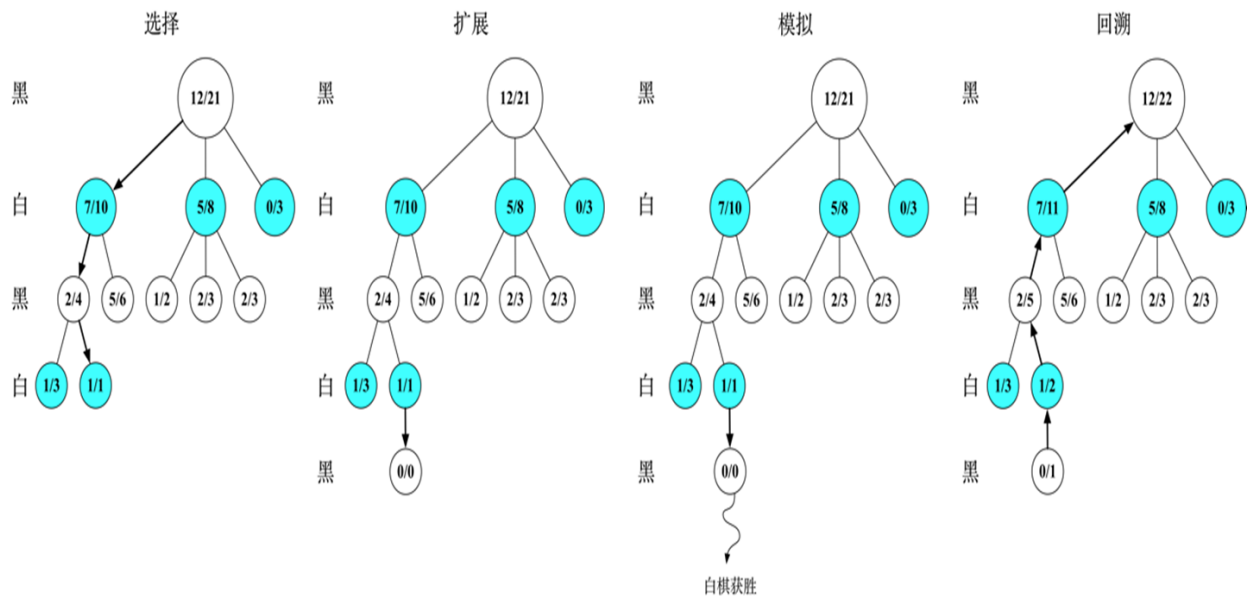
$$UCB1 = \overline{X}_j + C \times \sqrt{\frac{2 \ln n}{n_j}}$$

$\overline{X}_j$ 是第 j 个赌博机在过去时间内所获得的平均奖赏值, n_j 是在过去时间内拉动第 j 个赌博机臂膀的总次数, n 是过去时间内拉动所有赌博机臂膀的总次数。 C 是一个平衡因子, 其决定着在选择时偏重探索还是利用。

蒙特卡洛树搜索包括了四个步骤:

- 选择:从根节点开始,递归选择子节点,直至到达叶节点或者到达具有还未被扩展过的子节点的节点L.具体来说,使用UCB1最高的点.
- 扩展:如果L不是一个终止节点,则随机创建其后的一个未被访问节点,选择该节点作为后续子节点C.
- 模拟:从节点C出发,对游戏进行模拟,直到游戏结束.
- 反向传播:用模拟所得结果来回溯更新导致这个结果的每个节点中获胜次数和访问次数.

在蒙特卡洛树搜索中用到了两种策略学习机制:搜索树策略和模拟策略



假设由黑棋行棋,在选择阶段计算第二行三个节点的UCB1(假设C值为 $\sqrt{2}/2$)值为

$$\text{左1} = \frac{7}{10} + \sqrt{\frac{\ln 21}{10}} = 1.252$$

$$\text{左2} = \frac{5}{8} + \sqrt{\frac{\ln 21}{8}} = 1.243$$

$$\text{左3} = \frac{0}{3} + \sqrt{\frac{\ln 21}{3}} = 1.007$$

黑棋会选择左1.接下来由白棋行棋,对于白棋而言第一项为1-A/B,计算的UCB1为

$$\text{左1} = \frac{2}{4} + \sqrt{\frac{\ln 10}{10}} = 1.26$$

$$\text{左2} = \frac{1}{6} + \sqrt{\frac{\ln 10}{6}} = 0.786$$

白棋会选择左1.

四、监督学习

机器学习基本概念

机器学习分为监督学习、无监督学习、强化学习

监督学习的三个重要元素：标注数据、学习模型、损失函数

常见的损失函数：0-1损失函数、平方损失函数、绝对损失函数、对数损失函数/对数似然损失函数

损失函数名称	损失函数定义
0-1损失函数	$Loss(y_i, f(x_i)) = \begin{cases} 1, f(x_i) \neq y_i \\ 0, f(x_i) = y_i \end{cases}$
平方损失函数	$Loss(y_i, f(x_i)) = (y_i - f(x_i))^2$
绝对损失函数	$Loss(y_i, f(x_i)) = y_i - f(x_i) $
对数损失函数/对数似然损失函数	$Loss(y_i, P(y_i x_i)) = -\log P(y_i x_i)$

回归分析

一元线性回归

回归模型: $y = ax + b$

误差: $L(a, b) = \sum_{i=1}^n (y_i - ax_i - b)^2$

目标: 寻找一组a和b, 使得总误差和 $L(a, b)$ 值最小。在线性回归中, 解决如此目标的办法叫最小二乘法。

公式:

$$a = \frac{\sum_{i=0}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=0}^n x_i x_i - n \bar{x}^2}, b = \bar{y} - a \bar{x}$$

多元线性回归

回归模型: $f(x_i) = a_0 + \sum_{j=1}^D a_j x_{i,j} = a_0 + \mathbf{a}^T x_i$

误差: 均方误差

公式: $\mathbf{a} = (X X^T)^{-1} X y$

逻辑回归/对数几率回归

逻辑回归就是在回归模型中引入sigmoid函数的一种非线性的回归模型。

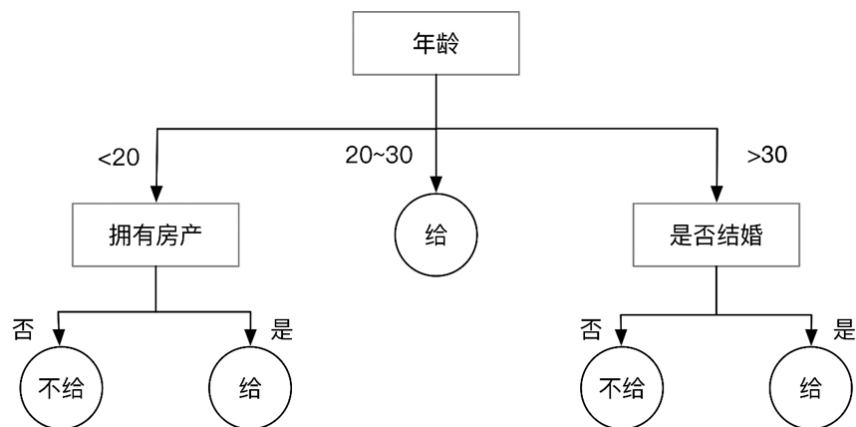
回归模型:

$$y = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

其中 $y \in (0, 1), z = \mathbf{w}^T \mathbf{x} + b$

决策树

决策树是一种通过树形结构来进行分类的方法。在决策树中, 树形结构中每个非叶子节点表示对分类目标在某个属性上的一个判断, 每个分支代表基于该属性做出的一个判断, 最后树形结构中每个叶子节点代表一种分类结果, 所以决策树可以看成是一系列以叶子节点为输出的决策规则。



信息熵

假设有 K 个信息，其组成了样本集合 D ，记第 k 个信息发生的概率为 $p_k (1 \leq k \leq K)$ 。如下定义这 K 个信息的信息熵：

$$E(D) = - \sum_{k=1}^K p_k \log_2 p_k$$

$E(D)$ 值越小，表示 D 包含的信息越确定，也称 D 的纯度越高。需要指出，所有 p_k 累加起来的和为1。

另一种更简单的度量指标是Gini系数：

$$Gini(D) = 1 - \sum_{k=1}^K p_k^2$$

线性判别分析 (LDA)

一种基于监督学习的降维方法，也称为Fisher线性判别分析 (FDA)。

对线性判别分析的降维步骤描述如下：

1. 计算数据样本集中每个类别样本的均值
2. 计算类内散度矩阵 S_w 和类间散度矩阵 S_b
3. 根据 $S_w^{-1} S_b W = \lambda W$ 来求解 $S_w^{-1} S_b$ 所对应前 r 个最大特征值所对应特征向量 $(w_1, w_2, \dots, w_r)$ ，构成矩阵 W
4. 通过矩阵 W 将每个样本映射到低维空间，实现特征降维

Ada Boosting

两个核心问题：

- 在每个弱分类器学习过程中，如何改变训练数据的权重
- 如何将一系列弱分类器组合成强分类器

训练过程

给定包含N个标注数据的训练集合

$$\Gamma = \{(x_1, y_1), \dots, (x_N, y_N)\}, x_i (1 \leq i \leq N) \in X \subseteq R^n, y_i \in Y = -1, 1$$

1. 初始化每个训练样本的权重 $D_1 = (w_{11}, \dots, w_{1N})$, 其中 $w_{1i} = \frac{1}{N} (1 \leq i \leq N)$
2. 迭代地利用加权样本训练弱分类器并增加错分类样本权重
 - 分别计算每个弱分类器的分类误差, 对于第m个弱分类器的误差计算如下, 其中 $I(\cdot)$ 为示性函数

$$err_m = \sum_{i=1}^N w_{mi} I(G_m(x_i) \neq y_i)$$

其实就是把该分类器分类不正确的样本权重求和

- 选择 err 最低的分类器 $G_m(x)$
 - 计算弱分类器 $G_m(x)$ 权重: $\alpha_m = \frac{1}{2} \ln \frac{(1 - err_m)}{err_m}$
 - 更新样本的分布权重, 其中分类正确的样本 i 权重更新为 $\frac{1}{2(1 - err_m)} \times w_i$, 分类错误的样本 i 权重更新为 $\frac{1}{2err_m} \times w_i$
 - 重新计算每个弱分类器的分类误差, 重复上述步骤
3. 以线性加权形式来组合弱分类器
 - 得到的强分类器 $G(x) = \text{sign}(\sum_{i=1}^M \alpha_m G_m(x))$

对于如下数据, 考虑使用Ada boosting方法来训练“是否出去玩”强分类器。每个弱分类器可考虑对单个属性的分类, 比如对于“心情指数”这一属性, 可考虑心情指数>2和心情指数<4两个方面。请问答下列问题:

- (1) Ada boosting在第一轮迭代中将会选择哪一个弱分类器?
- (2) 第一轮迭代前与迭代后每个样本的权重是多少?
- (3) 第二轮迭代选择的弱分类器是哪一个? 分类器权重是多少?
- (4) 写出三轮迭代后的强分类器的表达式(每个弱分类器可用字母替代)

序号	出去玩	天气状况	有同伴	零花钱	特殊节日	心情指数 (1 差-5 好)
1	是	好	无	多	是	5
2	是	一般	有	多	是	5
3	是	一般	有	少	否	1
4	是	一般	有	少	否	3
5	是	一般	有	少	否	5
6	是	好	无	多	是	5
7	是	好	无	多	是	5
8	否	一般	无	多	是	1
9	否	一般	有	少	否	1
10	否	一般	无	少	否	5

解析:

H_1 : 天气 H_2 : 同伴 H_3 : 钱 H_4 : 节目 H_5 : 心情

分变量	H_1 是 $\rightarrow$ 是 $H_1 \rightarrow 是$	H_2 是 $\rightarrow$ 是 $H_2 \rightarrow 是$	H_3 是 $\rightarrow$ 是 $H_3 \rightarrow 是$	H_4 是 $\rightarrow$ 是 $H_4 \rightarrow 是$	H_5 是 $\rightarrow$ 是 $H_5 \rightarrow 是$
err	$\frac{4}{10}$	$\frac{4}{10}$	$\frac{4}{10}$	$\frac{4}{10}$	$\frac{2}{10}$

2, 3, 4, 5 1, 6, 7, 9 3, 1, 5, 8 3, 4, 5, 8 3, 10

(1) 心情指数 > 2

(2) 前:

序号	1	2	3	4	5	6	7	8	9	10
权重	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$

$$\frac{1}{2(1-err)} \times \frac{1}{10} = \frac{1}{16}$$

$$\frac{1}{2err} \times \frac{1}{10} = \frac{1}{4}$$

后:

序号	1	2	3	4	5	6	7	8	9	10
权重	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{4}$	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{4}$

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1-err}{err} \right) = \frac{1}{2} \ln \frac{0.8}{0.2} = \frac{1}{2} \ln 4$$

(3)

分变量	H_1	H_2	H_3	H_4	H_5
err	$\frac{7}{16}$	$\frac{4}{16}$	$\frac{7}{16}$	$\frac{7}{16}$	$\frac{1}{2}$

选择 H_2

$$\alpha_2 = \frac{1}{2} \ln \left(\frac{1-\frac{4}{16}}{\frac{4}{16}} \right) = \frac{1}{2} \ln 3$$

$$\frac{1}{2(1-\frac{4}{16})} = \frac{2}{3}$$

$$\frac{1}{2 \times \frac{4}{16}} = 2$$

(4)

序号	1	2	3	4	5	6	7	8	9	10
权重	$\frac{1}{8}$	$\frac{1}{24}$	$\frac{1}{6}$	$\frac{1}{24}$	$\frac{1}{24}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{24}$	$\frac{1}{8}$	$\frac{1}{6}$

分变量	H_1	H_2	H_3	H_4	H_5
err	$\frac{7}{24}$	$\frac{1}{2}$	$\frac{7}{24}$	$\frac{7}{24}$	$\frac{1}{3}$

$$\text{选择 } H_1, \alpha_3 = \frac{1}{2} \ln \left(\frac{1-\frac{7}{24}}{\frac{7}{24}} \right) = \frac{1}{2} \ln \frac{17}{7}$$

$$\text{表达式为 } \text{sign} \left(\frac{1}{2} \ln 4 \cdot H_5 + \frac{1}{2} \ln 3 \cdot H_2 + \frac{1}{2} \ln \frac{17}{7} \cdot H_3 \right)$$

支持向量机(SVM)

传统识别理论认为，算法模型性能可用从训练样本集所得经验风险来衡量。

支持向量机认为：分类器对未知数据（即测试数据）进行分类时所产生的期望风险（即真实风险）不是由经验风险单独决定的，而是由两部分组成：

1. 从训练集合数据所得经验风险（如果经验风险小、期望风险很大，则是过学习）。
2. 置信风险，它与分类器的 VC 维及训练样本数目有关。

支持向量机把数据映射到线性可分的高维空间，用高维空间映射解决线性不可分问题。

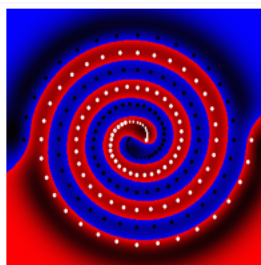
- 硬间隔：存在一个线性超平面将不同类别样本完全隔开。
- 软间隔：允许部分错分给定的训练样本。

映射采用的函数称为核函数。

图中的特征映射采用了那种核函数？

- ☐ A 线性
- ☐ B 多项式
- ☒ C 径向基
- ☐ D Sigmoid

提交



线性	$x_i^T x_j$
多项式	$(\gamma x_i^T x_j + c)^n$
径向基	$e^{-\frac{\ x_i - x_j\ ^2}{2\sigma^2}}$
Sigmoid	$\tanh(x_i, x_j - \gamma)$

生成学习模型

生成学习方法从数据中学习联合概率分布 $P(X, C)$ ，然后求出条件概率分布 $P(C|X)$ 作为预测模型，即
$$P(c_i|x) = \frac{P(x, c_i)}{P(x)}$$

$$\overbrace{P(c_i|x)}^{\text{后验概率}} = \frac{\overbrace{P(x, c_i)}^{\text{联合概率}}}{P(x)} = \frac{\overbrace{P(x|c_i)}^{\text{似然概率}} \times \overbrace{P(c_i)}^{\text{先验概率}}}{P(x)}$$

五、无监督学习

重要因素：数据特征和相似度函数

K均值聚类

步骤

1. 初始化聚类质心
 - 初始化 k 个聚类质心 $c = \{c_1, c_2, \dots, c_k\}$, $c_j \in R^m (1 \leq j \leq k)$
 - 每个聚类质心 c_j 所在集合记为 G_j

2. 将每个待聚类数据放入唯一一个聚类集合中

- 计算待聚类数据 x_i 和质心 c_j 之间的欧氏距离 $d(x_i, c_j)$ ($1 \leq i \leq n, 1 \leq j \leq k$)
- 将每个 x_i 放入与之距离最近聚类质心所在聚类集合中

3. 根据聚类结果，更新聚类质心

- 根据每个聚类集合中所包含的数据，更新该聚类集合质心值，即： $c_j = \frac{1}{|G_j|} \sum_{x_i \in G_j} x_i$

4. 算法循环迭代，直到满足条件

- 循环2、3步骤
- 终止条件：达到迭代次数上限，或者前后两次迭代中，聚类质心基本保持不变

缺点

- 需要事先确定聚类数目
- 需要初始化聚类中心
- 算法是迭代执行，时间开销非常大
- 欧式距离假设数据每个维度之间的重要性是一样的

主成分分析 (PCA)

主成分分析是一种特征降维方法。

降维后的结果要保持原始数据固有结构。

方差与协方差

方差等于各个数据与样本均值之差的平方之和之平均数。方差描述了样本数据的波动程度。

协方差

$$\text{cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - E(X))(y_i - E(Y))$$

协方差用于衡量两个变量之间的相关度：

- 当协方差 $\text{cov}(X, Y) > 0$ 时，称 X 与 Y 正相关
- 当协方差 $\text{cov}(X, Y) < 0$ 时，称 X 与 Y 负相关
- 当协方差 $\text{cov}(X, Y) = 0$ 时，称 X 与 Y 不相关（线性意义下）

可以使用皮尔逊相关系数将两组变量之间的关联度规则到一定取值范围内：

$$\text{corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}} = \frac{\text{Cov}(X, Y)}{\sigma_x \sigma_y}$$

皮尔逊相关系数的性质如下：

- $|\text{corr}(X, Y)| \leq 1$
- $|\text{corr}(X, Y)|$ 越大，线性相关度越大
- $\text{corr}(X, Y) = 1$ 的充要条件是存在常数 a 和 b ，使得 $Y = aX + b$
- $|\text{corr}(X, Y)| = 0$ 表示两者不存在线性相关关系（可能存在其他非线性相关的关系）

- $corr(X, Y) = corr(Y, X)$
- 正线性相关意味着变量 X 增加的情况下，变量 Y 也随之增加；负线性相关意味着变量 X 减少的情况下，变量 Y 随之增加。

相关性与独立性：

- 如果 X 和 Y 的线性不相关，则 $|corr(X, Y)| = 0$
- 如果 X 和 Y 的彼此独立，则一定 $|corr(X, Y)| = 0$ ，且 X 和 Y 不存在任何线性或非线性关系

算法动机

在降维之中，需要尽可能将数据向方差最大方向进行投影，使得数据所蕴含信息没有丢失，彰显个性。

主成分分析是将 n 维特征数据映射到 l 维空间($n \gg l$)，去除原始数据之间的冗余性。

将每个数据从 n 维高维空间映射到 l 维低维空间，每个数据所得到最好的 l 维特征就是使得每一维上样本方差都尽可能大。

算法描述

输入： n 个 d 维样本数据所构成的矩阵 X ，降维后的维数 l

输出：映射矩阵 $W = \{w_1, w_2, \dots, w_l\}$

使用拉格朗日乘子法将带约束的最优化问题转化为无约束最优化问题

特征人脸方法（了解）

特征人脸方法是一种应用主成分分析来实现人脸图像降维的方法，其本质是用一种称为“特征人脸”的特征向量按照线性组合形式来表达每一张原始人脸图像，进而实现人脸识别。

算法描述

- 将每幅人脸图像转换成列向量，如将一幅 32×32 的人脸图像转换成 1024×1 的列向量
- 输入： n 个 1024 维样本数据所构成的矩阵 X ，降维后的维数 l
- 输出：映射矩阵 $W = \{w_1, w_2, \dots, w_l\}$ （其中 $w_j (1 \leq j \leq l)$ 是特征人脸）

人脸表达的方法对比：聚类，主成分分析，非负矩阵分解

期望最大化(EM)算法（了解）

EM算法是一种重要的用于解决含有隐变量问题的参数估计方法。

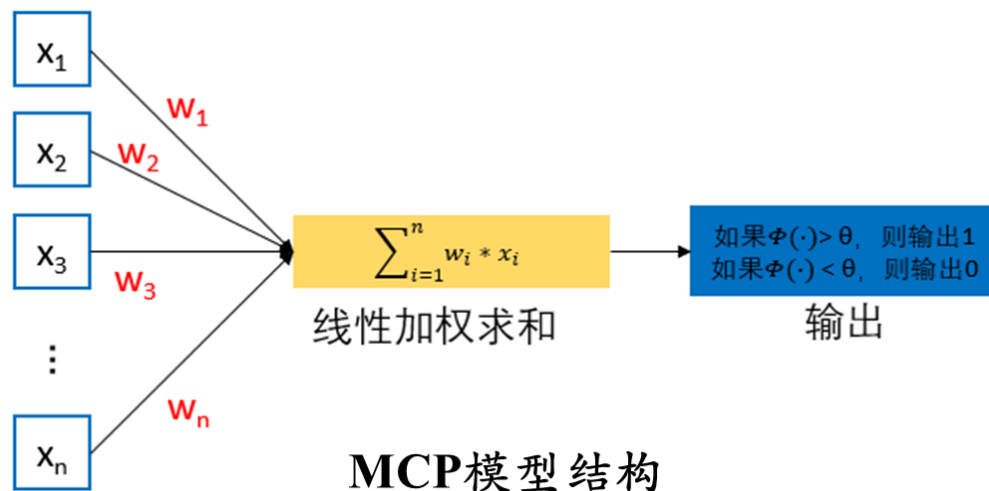
EM算法分为求取期望（E步骤）和期望最大化（M步骤）两个步骤。

- E步骤：假设模型参数的初始值，估计隐变量的取值
- M步骤：基于观测数据、模型参数和隐变量取值一起最大化拟合数据，更新模型参数
- 得到新的隐变量，如此迭代

六、深度学习

深度学习历史发展

神经科学家Warren McCulloch和逻辑学家Walter Pitts合作提出了"McCuiioch-Pitts(MCP) neuron"的思想。MCP是最早的神经网络雏形。



神经网络研究的突破来自于Frank Rosenblatt在20世纪50年代所提出的“感知机（perceptron）”模型。

前馈神经网络

神经元是深度学习中的基本单位，每个神经元可以分为三部分：输入、激活函数、输出

- 输入：对相邻前向神经元输入信息进行加权累加： $In = \sum_{i=1}^n w_i \times a_i$
- 激活函数：对累加结果进行非线性变换（通过激活函数）： $g(x)$
- 输出：神经元的输出： $Out = g(In)$

常用的激活函数：对输入信息进行非线性变换

注意这里的Sigmoid和Tanh函数的导数中是 $f(x)$ 而不是 x ，在计算的时候不能搞错

激活函数名称	函数功能	函数图像	函数求导
Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$		$f'(x) = f(x)(1 - f(x))$
Tanh	$f(x) = \frac{2}{1 + e^{-2x}} - 1$		$f'(x) = 1 - f(x)^2$
<u>Relu</u> (Rectified Linear Unit)	$f(x) = \begin{cases} 0, & \text{for } x < 0 \\ x, & \text{for } x \geq 0 \end{cases}$		$f'(x) = \begin{cases} 0, & \text{for } x < 0 \\ 1, & \text{for } x \geq 0 \end{cases}$

还有一个常用的激活函数为softmax函数

- softmax函数一半用于多分类问题

- 将输入 x_i 映射到第 i 个类别的概率: $y_i = softmax(x) = \frac{e^{x_i}}{\sum_{j=1}^k e^{x_j}}$
- $0 < y_i < 1, \sum_i y_i = 1$, 可将输出概率最大的作为分类目标

损失函数又称代价函数, 用来计算模型预测值与实际值之间的误差。在训练过程中可根据损失函数来计算神经网络的误差大小, 进而优化神经网络训练参数。

两种最常用损失函数:

- 均方误差损失函数 (MSE), 定义为: $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- 交叉熵损失函数
 - 用来度量两个概率分布间的差异。假定 p 和 q 是数据 x 的两个概率分布, 通过 q 来表示 p 的交叉熵可如下计算:

$$H(p, q) = - \sum_x p(x) \times \log q(x)$$

- 刻画了两个概率分布之间的距离, 旨在描绘通过概率分布 q 来表达概率分布 p 的困难程度。根据公式不难理解, 交叉熵越小, 两个概率分布 p 和 q 越接近。

假设我们有一个三个样本的训练数据, 需要对它们进行二分类, 其中第一个样本 x 的类别为1, 其余样本的类别为2, 那么可以将实际的类别概率分布表示为 $y = (1, 0, 0)$ 。假设经过神经网络在Softmax函数映射之前的值为 (z_1, z_2, z_3) , 经过Softmax函数映射, 得到预测的类别分布概率 $\hat{y} = (\hat{y}_1, \hat{y}_2, \hat{y}_3)$ 。这里的 $\hat{y}_1, \hat{y}_2, \hat{y}_3$ 为 $(0, 1)$ 范围之间的一个概率值。由于样本 x 属于第一个类别, 因此希望神经网络所预测得到的 $\hat{y}_1$ 取值要远远大于 $\hat{y}_2$ 和 $\hat{y}_3$ 的取值。

训练中可利用如下交叉熵损失函数来对模型参数进行优化:

$$CrossEntropy = -(y_1 \times \log(\hat{y}_1) + y_2 \times \log(\hat{y}_2) + y_3 \times \log(\hat{y}_3))$$

由于上式中 y_2 和 y_3 均为0, y_1 为1, 因此损失函数简化为 $CrossEntropy = -\log(\hat{y}_1)$

接下来需要将误差从 $\hat{y}_1$ 传递给 z_1

$$\begin{aligned} \frac{\partial \hat{y}_1}{\partial z_1} &= \frac{\partial}{\partial z_1} \left(\frac{e^{z_1}}{\sum_k e^{z_k}} \right) = \frac{(e^{z_1})' \times \sum_k e^{z_k} - e^{z_1} \times e^{z_1}}{(\sum_k e^{z_k})^2} \\ &= \frac{e^{z_1}}{\sum_k e^{z_k}} - \frac{e^{z_1}}{\sum_k e^{z_k}} \times \frac{e^{z_1}}{\sum_k e^{z_k}} = \hat{y}_1(1 - \hat{y}_1) \end{aligned}$$

我们下面求交叉熵损失函数经过 $\hat{y}_1$ 对 z_1 的偏导数, 我们用 H 代替交叉熵, 用链式法则计算

$$\frac{\partial H}{\partial \hat{y}_1} \times \frac{\partial \hat{y}_1}{\partial z_1} = -\frac{1}{\hat{y}_1} \times \hat{y}_1(1 - \hat{y}_1) = \hat{y}_1 - 1$$

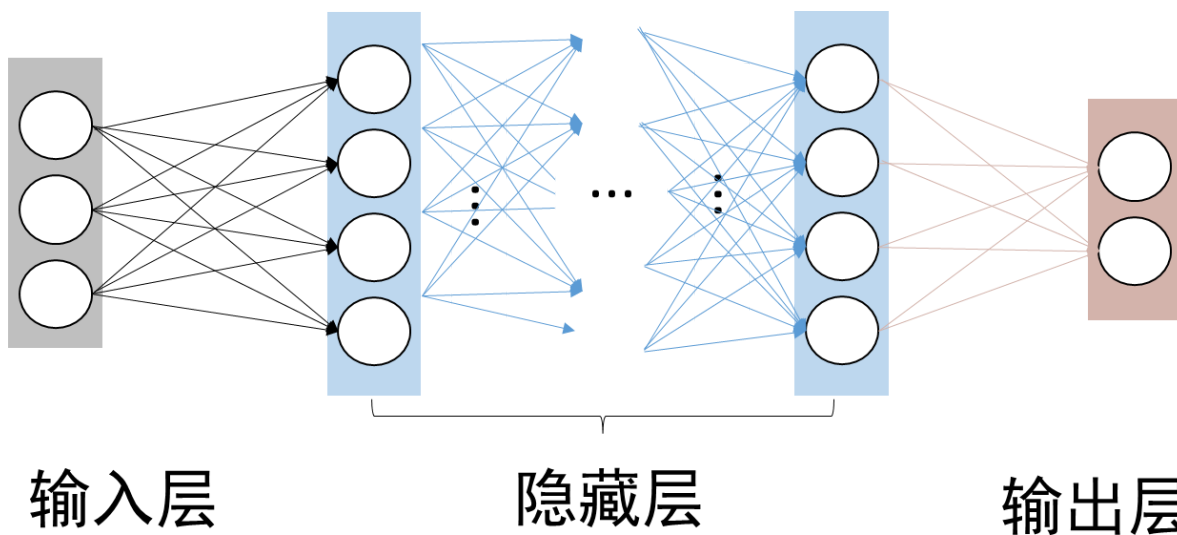
同样也可以计算出交叉熵损失函数经过 $\hat{y}_1$ 对 z_2 的偏导数

$$\frac{\partial H}{\partial \hat{y}_2} \times \frac{\partial \hat{y}_2}{\partial z_1} = -\frac{1}{\hat{y}_1} \times \hat{y}_1 \hat{y}_2 = -\hat{y}_2$$

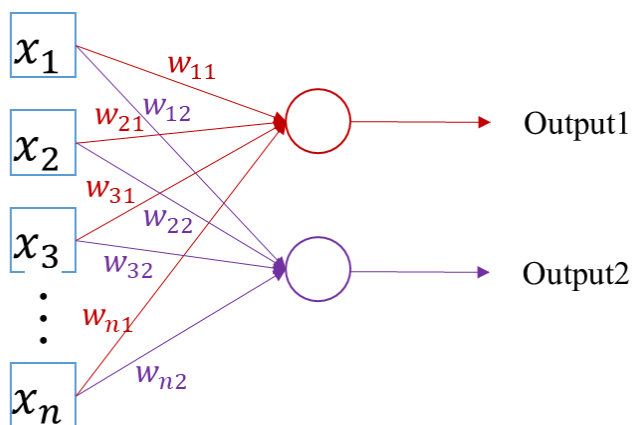
对 z_3 的推导同 z_2 。得到这些结果之后, 还可以继续通过链式法则将损失函数误差继续往输入端传播。

前馈神经网络:

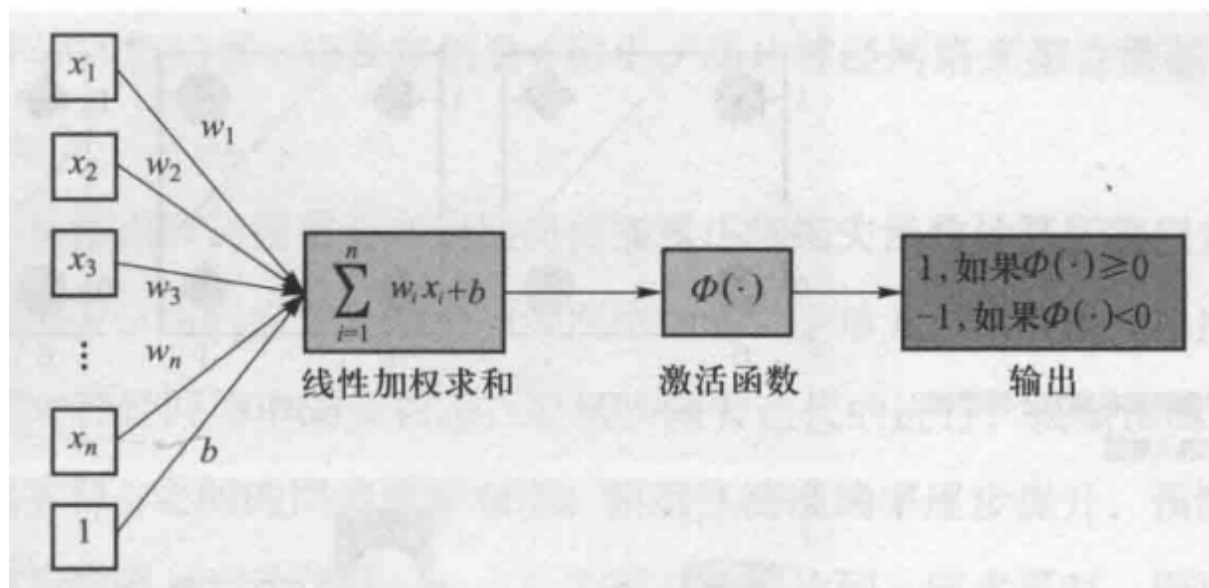
- 各神经元接受前一级输入并输出到下一级, 模型中没有反馈
- 层间“全连接”, 即两个相邻层之间的神经元完全成对连接, 但层内的神经元不相互连接。



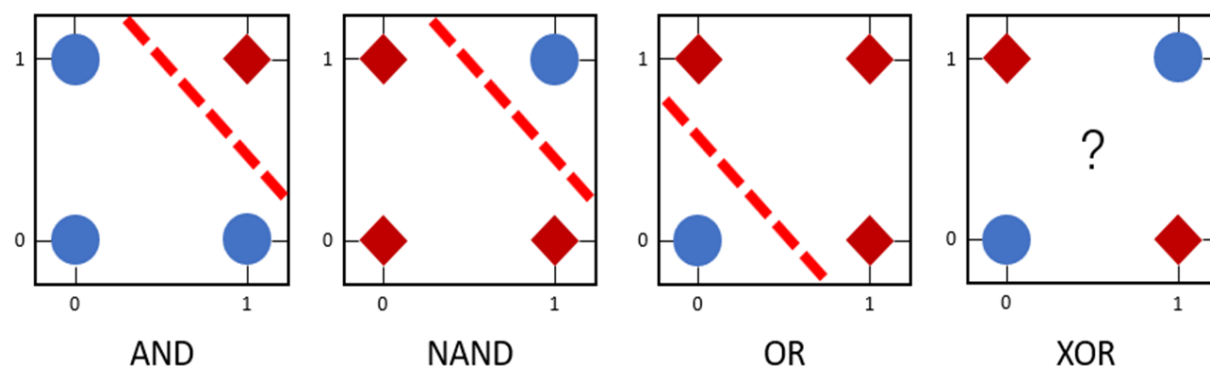
感知机神经网络是一种特殊的前馈神经网络，它没有隐藏层，只有输入层和输出层。这也就决定了它无法拟合复杂的数据。下图中的 $w_{ij}(1 \leq i \leq n, 1 \leq j \leq 2)$ 构成了感知机模型的参数。



早期的感知机结构和MCP模型相似，由一个输入层和一个输出层构成，因此也被称为“单层感知机”。输入层负责接受实数值的输入向量，输出层只能输出1或-1两个值。这里加权求和中有一个参数 b ，为偏置项。



单层感知机可以被用来区分线性可分数据，如下图中的逻辑与、逻辑与非、逻辑或都是线性可分的函数，都可以用感知机来模拟。但是逻辑异或是线性不可分的，也就是无法画出一条直线将取值不同的点分散在直线的两侧。



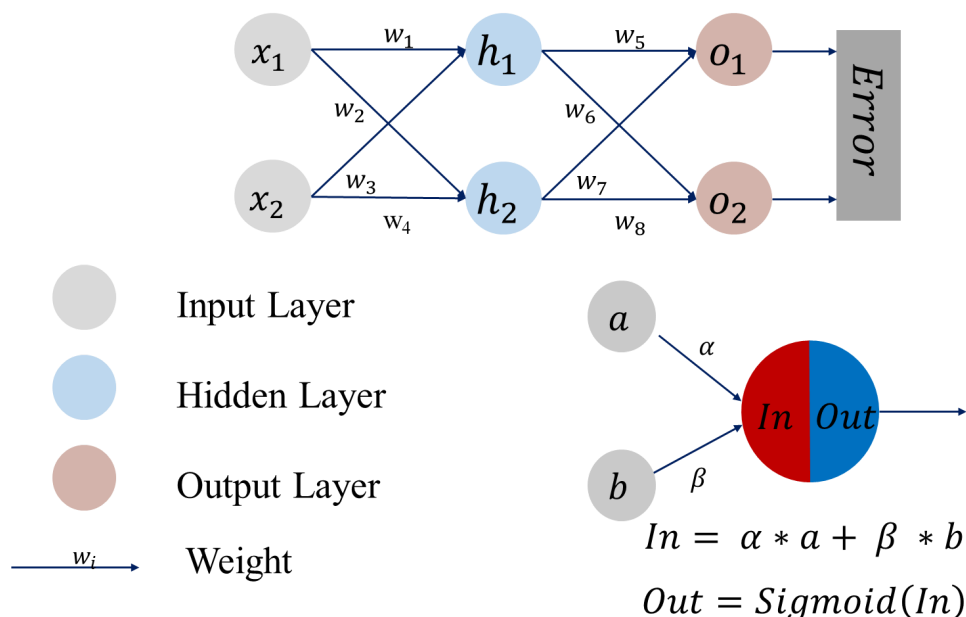
单层感知机模拟不同逻辑函数功能的示意图

多层感知机由输入层，输出层和至少一层的隐藏层构成，性质和前馈神经网络类似。

优化网络参数

为了使神经网络的效果更好，我们需要从标注数据出发，优化模型参数。我们使用评分函数将输入数据映射为类别置信度大小，损失函数来估计模型预测值与真实值之间的差距。损失函数给出的差距越小，则模型的鲁棒性就越好。常用的损失函数有softmax或者SVM。

可以使用梯度下降算法来使损失函数最小化。在多元函数中，梯度是对每一变量所求导数组成的向量。梯度的反方向是函数值下降最快的方向，因此是求解的方向。



以上面这个神经网络为例，分析一下前向传播和误差反向传播的过程。

以 h_1 和 h_2 为例，有如下函数关系

$$In_{h_1} = w_1 \times x_1 + w_3 \times x_2$$

$$h_1 = Out_{h_1} = Sigmoid(In_{h_1})$$

$$In_{o_1} = w_5 \times h_1 + w_7 \times h_2$$

$$o_1 = Out_{o_1} = Sigmoid(In_{o_1})$$

损失函数采取均方差误差，为

$$Error = \frac{1}{2} \sum_{i=1}^2 (o_i - y_i)^2$$

以 w_5 和 w_1 为例，计算梯度

在 w_5 中，由于 $Error$ 与 w_5 只通过 o_1 建立关系，所以可以得到

$$\delta_5 = \frac{\partial Error}{\partial w_5} = \frac{\partial Error}{\partial o_1} \times \frac{\partial o_1}{\partial In_{o_1}} \times \frac{\partial In_{o_1}}{\partial w_5}$$

其中第一项的值可以直接由 $Error$ 对 o_1 求偏导得到，第二项的值可以由Sigmoid函数的求导公式结合 o_1 的计算公式得到，注意这里第二项是 $o_1(1 - o_1)$ ，第三项可以由 In_{o_1} 的计算公式得到。于是计算上式

$$\delta = (o_1 - y_1) \times (o_1(1 - o_1)) \times h_1$$

在计算 w_1 时，由于 w_1 是 h_1 的参数，它通过 o_1 和 o_2 均与 $Error$ 建立起了联系，所以计算 w_1 的梯度要分成两部分之和，具体计算公式为

$$\begin{aligned} \delta_1 &= \frac{\partial Error}{\partial w_1} = \frac{\partial Error}{\partial o_1} \times \frac{\partial o_1}{\partial w_1} + \frac{\partial Error}{\partial o_2} \times \frac{\partial o_2}{\partial w_1} \\ &= \frac{\partial Error}{\partial o_1} \times \frac{\partial o_1}{\partial In_{o_1}} \times \frac{\partial In_{o_1}}{\partial h_1} \times \frac{\partial h_1}{\partial In_{h_1}} \times \frac{\partial In_{h_1}}{\partial w_1} + \frac{\partial Error}{\partial o_2} \times \frac{\partial o_2}{\partial In_{o_2}} \times \frac{\partial In_{o_2}}{\partial h_1} \times \frac{\partial h_1}{\partial In_{h_1}} \times \frac{\partial In_{h_1}}{\partial w_1} \\ &= \left(\frac{\partial Error}{\partial o_1} \times \frac{\partial o_1}{\partial In_{o_1}} \times \frac{\partial In_{o_1}}{\partial h_1} + \frac{\partial Error}{\partial o_2} \times \frac{\partial o_2}{\partial In_{o_2}} \times \frac{\partial In_{o_2}}{\partial h_1} \right) \times \frac{\partial h_1}{\partial In_{h_1}} \times \frac{\partial In_{h_1}}{\partial w_1} \\ &= \left(\frac{\partial Error}{\partial o_1} \times \frac{\partial o_1}{\partial In_{o_1}} \times w_5 + \frac{\partial Error}{\partial o_2} \times \frac{\partial o_2}{\partial In_{o_2}} \times w_6 \right) \times \frac{\partial h_1}{\partial In_{h_1}} \times \frac{\partial In_{h_1}}{\partial w_1} \end{aligned}$$

之后再继续计算其余偏导数即可。

最后需要对参数进行更新，其中 η 被称为学习率

$$w'_i = w_i - \eta \times \delta_i$$

卷积神经网络

在前馈神经网络中，输入层的输入直接与第一个隐藏层中所有神经元相互连接。如果输出的是一张尺寸较大的图像，这样输入层到第一个隐藏的的参数会特别多，这样不仅占用计算机内存，也使得模型更加难以收敛。

因此，在连接隐藏层之前，对图像进行卷积操作，降低分辨率，可以减少输入的参数。通过卷积操作得到的图像称为特征图。

下面通过一个例子展示卷积的计算过程。

给定输入信号和卷积核，当步长为2时输出各个输出h的表达式为？

a	b	c	d	e
f	g	h	i	j
k	l	m	n	o
p	q	r	s	t
u	v	w	x	y

输入

w1	w2	w3
w4	w5	w6
w7	w8	w9

卷积核

h1	h2
h3	h4

输出

步长为2，就是每次向下或者向右平移时移动两个单位。卷积操作就是将对应位置的像素值与卷积核对应位置的权重相乘再求和，所以计算结果为

$$h_1 = w_1 \times a + w_2 \times b + w_3 \times c + w_4 \times f + w_5 \times g + w_6 \times h + w_7 \times k + w_8 \times l + w_9 \times m$$

$$h_2 = w_1 \times c + w_2 \times d + w_3 \times e + w_4 \times h + w_5 \times i + w_6 \times j + w_7 \times m + w_8 \times n + w_9 \times o$$

$$h_3 = w_1 \times k + w_2 \times l + w_3 \times m + w_4 \times p + w_5 \times q + w_6 \times r + w_7 \times u + w_8 \times v + w_9 \times w$$

$$h_4 = w_1 \times m + w_2 \times n + w_3 \times o + w_4 \times r + w_5 \times s + w_6 \times t + w_7 \times w + w_8 \times x + w_9 \times y$$

现在这里的输入的大小为 5×5 ，如果这里的输入改成 $5 \times 5 \times 3$ ，卷积核的大小也需要改为 $3 \times 3 \times 3$ ，但是输出依旧是 2×2 ，这时的输出里的每一个结果就是需要将27个位置对应的数相乘再相加。如果需要得到多个输出，可以使用多个卷积核分别进行卷积操作，比如使用4个不同的卷积核，输出的大小就变为 $2 \times 2 \times 4$ 。

有的时候，我们希望对于位于卷积图像边缘位置的像素点进行卷积滤波，因此提出了一种padding的技术，就是在边缘像素点周围填充“0”。

在对原始图像做卷积操作后，可以使用ReLU激活函数对卷积后结果进行处理，也就是将得到的特征图中的负数转换为0。

卷积所得结果中，每个输出点的取值仅依赖于输入图像中该点及其邻域区域点的取值，与区域之外的其他无关，该区域被称为感受野。感受野是CNN每一层输出的特征图上的一个点在输入图像上映射的区域。

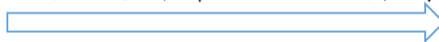
在卷积过程之后，可以进行池化操作，对特征图进行下采样，以获得最主要信息。

常用的池化操作有：

- 最大池化：从输入特征图中每一个区域寻找最大值。

1	5	4	3
2	6	4	0
3	1	7	9
2	8	6	5

对2*2大小区域，按照步长为2进行最大池化操作

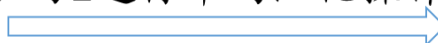


6	4
8	9

- 平均池化：对输入特征图的每一个区域的值求平均值（取整）

1	5	4	3
2	6	4	0
3	1	7	9
2	8	6	5

对2*2大小区域，按照步长为2进行平均池化操作



3	2
3	6

在对图像经过多次的卷积、激活函数、池化操作后，最后通过全连接层来学习得到输入数据的特征表达，即分布式向量表达。

为了缓解神经网络在训练过程中出现的过拟合问题，需要采取一些正则化技术来提升神经网络的泛化能力。下面介绍两种正则化方法。

- L_1 范数：数学表示为 $\|W\|_1 = \sum_{i=1}^N |w_i|$ ，指模型参数 W 中各个元素绝对值之和。 L_1 范数也被称为“稀疏规则算子”。
- L_2 范数：数学表示为 $\|W\|_2 = \sqrt{\sum_{i=1}^N w_i^2}$ ，指模型参数 W 中各个元素平方和再开方。

循环神经网络（了解）

略

深度生成学习（了解）

略

深度学习应用（了解）

略

七、强化学习

强化学习问题定义

强化学习：在与环境交互中进行学习

在强化学习中，智能主体按照某种策略，根据当前状态选择合适的动作，并能获得一定的收益（奖励）。

	有监督学习	无监督学习	强化学习
学习依据	基于监督信息	基于对数据结构的假设	基于评价(evaluative)
数据来源	一次性给定	一次性给定	在交互中产生
决策过程	单步(one-shot)	无	序列(sequential)
学习目标	样本到语义标签的映射	同一类数据的分布模式	选择能够获取最大收益的状态到动作的映射

为了介绍接下来的内容，我们基于一个简单的例子来介绍相关的定义。

在一个 3×3 的网格中，假设有一个机器人位于 s_1 ，试图从 s_1 这一初始位置向 s_9 这一目标位置移动，假设机器人每一步只能向上或向右移动一个方格，到达目标位置 s_9 则会获得奖励且游戏终止，机器人在移动过程中如果越出方格则会被惩罚，并且游戏终止。那么，如何学习一种策略能够帮助机器人从 s_1 走到 s_9 ？

在这个问题中，智能主体为迷宫机器人，环境是 3×3 的网格，状态是机器人当前所处的方格，状态的取值范围为 $\{s_1, s_2, \dots, s_9, s_d\}$ ，其中 s_d 表示机器人出界的情况。机器人每次可采取的行动为向上或者向右移动一个方格，机器人所获得的奖励包括他到达 s_9 时的正奖励，以及越界时得到的负奖励（惩罚），到达其他状态时不被奖励也不被惩罚。

在这个例子中，我们可以采取随机过程机器人与环境之间的交互。

一个随机过程是一列随时间变化的随机变量。当时间是离散量时，一个随机过程可以表示为 $\{X_i\}_{i=0,1,2,\dots}$ ，这里的每一个 X_i 都是一个随机过程，这被称为离散随机过程。我们可以将这个问题视为一个马尔可夫性的离散随机过程，即满足

$$P(X_{t+1} = x_{t+1} | X_0 = x_0, X_1 = x_1, \dots, X_t = x_t) = p(X_{t+1} = x_{t+1} | X_t = x_t)$$

可以理解成 $t + 1$ 时刻的状态只与 t 时刻的状态有关。

定义离散马尔可夫过程 $\{S_t\}_{t=0,1,2,\dots}$ ，由于其满足马尔可夫性，因此可以定义状态转移概率 $P(S_{t+1} | S_t)$ 。

我们将到达 s_9 时的奖励定为1，出界时的惩罚值为-1，其他情况下奖励值为0。为了比较不同奖励机制的优劣，在每个时刻定义回报来反映该时刻可以得到的累加奖励：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

其中折扣因子 $\gamma \in [0, 1]$ ， $R_{t+k} (t > 0)$ 表示 $t + k$ 时刻获得的奖励。

于是，我们得到了马尔可夫奖励过程，其形式化的定义为 $MRP = (S, P, R, \gamma)$ 。这个模型虽然能够用奖励和回报来刻画智能体的目标，但是仍然不能体现机器人的能动性，缺乏让机器人与环境进行交互的手段。

我们引入动作集合 A ，在这个问题中 $S = \{\text{上}, \text{右}\}$ ，状态转移概率与动作有关因此将状态转移概率重新定义为 $P(S_{t+1} | S_t, a_t)$ ， $a_t \in A$ 。奖励也有可能受到动作的影响，因此修改奖励函数为 S_t, a_t, S_{t+1} 。现在就可以通过 $MDP(S, A, P, R, \gamma)$ 来刻画马尔可夫决策过程。

马尔可夫过程中产生的状态序列称为轨迹，轨迹的长度可以是无限的，也可以是有终止状态的。有终止状态的问题叫做分段的，否则叫做持续的。分段问题中，一个从初始状态到终止状态的完整轨迹称为一个片段或回合。

接下来我们对强化问题下定义。首先定义如下函数

- 价值函数 $V : S \mapsto \mathbb{R}$ ，其中 $V_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$ ，即在第 t 步状态为 s 时，按照策略 π 行动后在未来所获得的反馈值的数学期望
- 动作-价值函数 $q : S \times A \mapsto \mathbb{R}$ ，其中 $q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$ 表示在第 t 步状态为 s 时，按照策略 π 采取动作 a 后，在未来所获得的反馈值的期望

至此，强化学习转化为一个策略问题：寻找一个最优策略 π^* ，对任意 $s \in S$ 使得 $V_{\pi^*}(s)$ 值最大。

最后我们介绍贝尔曼方程，也称动态规划方程。

- 价值函数的贝尔曼方程

$$V_\pi(s) = \mathbb{E}_{a \sim \pi(s, \cdot)} \mathbb{E}_{S' \sim P(\cdot | s, a)} [R(s, a, s') + \gamma V_\pi(s')]$$

- 动作-价值函数的贝尔曼方程

$$q_{\pi}(s, a) = \mathbb{E}_{S' \sim P(\cdot | s, a)} [R(s, a, s') + \gamma \mathbb{E}_{a' \sim \pi(s', \cdot)} [q_{\pi}(s', a')]]$$

价值函数的贝尔曼方程描述了当前状态价值函数和其后续状态价值函数之间的关系，即当前状态价值函数等于瞬时奖励的期望加上后续状态的(折扣)价值函数的期望。而动作-价值函数的贝尔曼方程描述了当前动作-价值函数和其后续动作-价值函数之间的关系，即当前状态下的动作-价值函数等于瞬时奖励的期望加上后续状态的(折扣)动作-价值函数的期望。

基于价值的强化学习

策略优化定理：给定任意状态 $s \in S$ ，如果两个策略 π 和 π' 满足如下条件：

$$q_{\pi}(s, \pi'(s)) \geq q_{\pi}(s, \pi(s))$$

那么对于任意状态 $s \in S$ ，有

$$V_{\pi'}(s) \geq V_{\pi}(s)$$

即策略 π' 不比策略 π 差。

下面介绍在状态集合有限前提下三种常见的策略评估方法，它们分别是基于动态规划的方法、基于蒙特卡洛采样的方法和时序差分法。

- 基于动态规划的方法：使用迭代的方法求解贝尔曼方程组

缺点：

- 智能主体需要实现知道状态转移概率
- 无法处理状态集合大小无限的情况

- 蒙特卡洛采样：选择不同的起始状态，按照当前策略 π 采样若干轨迹，记他们的集合为 D ，枚举 $s \in S$ 。计算 D 中 s 每次出现时对应的反馈 $G_1, G_2, \dots, G_k$ ， $V_{\pi}(s) \leftarrow \frac{1}{k} \sum_{i=1}^k G_i$

优点：

- 不必知道状态转移概率
- 容易扩展到无限状态集合的问题中

缺点：

- 状态集合比较大时，一个状态在轨迹可能非常稀疏，不利于估计期望
- 在实际问题中，最终反馈需要在终止状态才能知晓，导致反馈周期较长

- 时序差分法：通过采样 a 和 s' 来估计 $V_{\pi}(s)$ 的取值 $R(s, a, s') + \gamma V_{\pi}(s')$ ，并以 α 作为权重接受新的估计值，即把价值函数更新为 $V_{\pi}(s) + \alpha [R + \gamma V_{\pi}(s') - V_{\pi}(s)]$

Q学习算法：一种基于时序差分的算法。分为以下几个步骤：

1. 初始化 s 为初始状态
2. 比较动作-价值函数最优的动作，设为 a
3. 执行动作 a ，观察奖励 R 和下一个状态 s'
4. 更新 $q_{\pi}(s, a) \leftarrow q_{\pi}(s, a) + \alpha [R + \gamma \max_{a'} q_{\pi}(s', a') - q_{\pi}(s, a)]$
5. 更新状态 $s \leftarrow s'$ ，重复执行步骤2到5，直到 s 是终止状态（一个片段）
6. 重复执行步骤1到6，直到 q_{π} 收敛

这样的Q学习算法也会收敛到非最优策略，因为他选择下一步策略总是选择目前已知最优的策略（称为利用），缺乏探索性。

在题 6 中，若图 2 表示算法的初始状态，其中 a/b 表示对应状态的动作-价值函数的取值，斜线左侧的 a 表示 $q_{\pi}(s, \text{上})$ ，斜线右侧的 b 表示 $q_{\pi}(s, \text{右})$ 。若 $\alpha = 0.5$ ，试给出算法 7.6 中的 Q 学习算法的一个片段的执行过程，并给出执行完该片段后每个状态的策略。

0.1/0	0/0
0.1/0	0.1/0

图 2 Q 学习算法的初始状态

解 根据算法 7.2.6 中的 Q 学习算法， s_1 为初始状态，根据当前策略求出智能体应该采取的动作 $a = \operatorname{argmax}_a q_{\pi}(s_1, a) = \text{上}$ ，执行这个动作，得到奖励 $R = 0$ 和进入下一状态 $s' = s_3$ ，因此可如下更新对应的动作-价值函数：

$$\begin{aligned} q_{\pi}(s_1, \text{上}) &\leftarrow q_{\pi}(s_1, \text{上}) + \alpha [R + \gamma \max_a q_{\pi}(s', a') - q_{\pi}(s, a)] \\ &= 0.1 + 0.5 \times [0 + 0.99 \times \max\{0, 0.1\} - 0.1] = 0.0995 \end{aligned}$$

此时的 q 函数为：

0.1/0	0/0
0.0995/0	0.1/0

同时令当前状态为 s_3 ，此时智能体应该采取的动作 $a = \operatorname{argmax}_a q_\pi(s_3, a) = \text{上}$ ，执行这个动作，得到奖励 $R = -1$ 和进入下一状态 $s' = s_d$ ，因此可如下更新对应的动作-价值函数：

$$q_\pi(s_3, \text{上}) \leftarrow q_\pi(s_3, \text{上}) + \alpha [R + \gamma \max_a q_\pi(s', a) - q_\pi(s, a)]$$

$$= 0.1 + 0.5 \times [-1 + 0.99 \times \max\{0, 0\} - 0.1] = -0.45$$

此时算法达到终止状态 s_d ，该片段结束。此时的 q 函数为：

-0.45/0	0/0
0.0995/0	0.1/0

此时每个状态的策略为：

→	
↑	↑

ϵ 贪心策略：以 $1 - \epsilon$ 的概率执行最优的下一步动作，以 ϵ 的概率随机选择下一步动作。

注意这时采样时策略为 ϵ 贪心策略，但是更新 $q_\pi(s, a)$ 是里面计算采用的依然是策略最优的操作。像这样更新时的目标策略与采样策略不同的方法，叫做离策略方法。

将 q 函数参数化，用一个非线性回归模型来拟合 q 函数，例如（深度）神经网络，这使得算法称为深度 Q 学习，优点如下

- 能够用有限的参数刻画无限的状态
- 由于回归函数的连续性，没有探索过的状态也可通过周围的状态来估计

基于策略的强化学习（了解）

略

深度强化学习的应用（了解）

略

八、博弈

博弈相关概念

标志着现代博弈理论的初步形成的事件是：1944年冯·诺伊曼与奥斯卡·摩根斯特恩合著《博弈论与经济行为》的出版

囚徒困境：警方逮捕了共同犯罪的甲和乙，但没有掌握充分证据，将他们分开审讯，并告诉他们如下事实：

- 若一人认罪并指证对方，而另一方保持沉默，则此人会被当即释放，沉默者会被判监禁10年
- 若两人都沉默，则根据已有的犯罪证据两人各判半年
- 若两人都认罪并相互指证，则两人各判5年

(甲,乙) 收益	乙沉默 (合作)	乙认罪 (背叛)
甲沉默 (合作)	(-0.5, -0.5)	(-10, 0)
甲认罪 (背叛)	(0, -10)	(-5, -5)

在囚徒困境中，最优解为两个人同时沉默，但是两个人实际倾向于**选择同时认罪**（均衡解）。

博弈的分类：

- 合作博弈与非合作博弈
- 静态博弈和动态博弈
- 完全信息博弈与不完全信息博弈

纳什均衡：任何玩家单独改变策略都不会得到好处。

纳什均衡的本质：不后悔

遗憾最小化算法

若干定义

- 玩家 i 所采用的策略为 σ_i ，一个策略组 σ 包含所有玩家的策略， $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_{|N|})$
 - 玩家 i 的策略空间用 Σ_i 表示
 - σ_{-i} 表示 σ 中除了 σ_i 之外的策略
- 玩家 i 在给定策略 σ 下的期望收益为： $\mu_i(\sigma)$

策略选择

根据过去博弈中的遗憾程度来决定动作选择的方法

玩家 i 在过去 T 轮中采取策略 σ_i 的累加遗憾值为

$$Regret_i^T(\sigma_i) = \sum_{t=1}^T (\mu_i(\sigma_i, \sigma_{-i}^t) - \mu_i(\sigma^t))$$

在第 $T + 1$ 轮次玩家 i 选择策略 a 的概率如下（悔值越大越选择）

$$P(a) = \frac{Regret_i^T(a)}{\sum_{b \in \Sigma_i} Regret_i^T(b)}$$

遗憾最小化算法：石头-剪刀-布的例子

- 假设玩家A和B进行石头-剪刀-布 (Rock-Paper-Scissors) 的游戏
 - 玩家收益：获胜1分，失败-1分，平局0分
- 第一局时，若玩家A出石头 (R)，玩家B出布 (P)
 - 此时玩家A的收益 $\mu_A(R, P) = -1$ ，玩家B的收益为 $\mu_B(P, R) = 1$
 - 如果玩家A选择出布或剪刀，则收益值为 $\mu_A(P, P) = 0$ 或 $\mu_A(S, P) = 1$
- 玩家A第一局没有出布的遗憾值为 $\mu_A(P, P) - \mu_A(R, P) = 1$
 - 没有出剪刀的遗憾值为 $\mu_A(S, P) - \mu_A(R, P) = 2$
- 在第二局中，A选择R, P, S这三个策略的概率分别为0, 1/3, 2/3

遗憾最小化算法：石头-剪刀-布的例子

- 玩家A每一轮遗憾值及第二轮后的累加遗憾取值：
 - 前提：在第一轮中玩家A选择石头和B选择布、在第二局中玩家A选择剪刀和玩家B选择石头情况下

每轮悔值\策略	石头	剪刀	布
第一轮悔值	0	2	1
第二轮悔值	1	0	2
$Regret_A^2$	1	2	3

- 在第三局时，玩家A选择R, S, P的概率分别为1/6、2/6、3/6
- 在实际使用中，可以通过多次模拟迭代累加遗憾值找到每个玩家在每一轮次的最优策略。但是当博弈状态空间呈指数增长时，对一个规模巨大的博弈树无法采用最小遗憾算法==>虚拟最小遗憾算法

尝试计算在二轮中，假设玩家A在第二局按照遗憾值最大的策略选择“剪刀”、玩家B随机选择“布”，游戏结束后，求玩家B不同行动的遗憾值以及在第三轮中可能采取的每种行动的概率。

玩家 B 在两轮博弈后，遗憾值如下表：

玩家 B 在两轮后所得到的遗憾值			
遗憾值\策略	石头	剪刀	布
第一轮	-1	-2	0
第二轮	2	1	0
$Regret_A^2$	1	-1	0
$Regret_A^{2,+}$	1	0	0

按照遗憾匹配算法，第二轮过后，玩家 B 选择出石头的概率为 $\frac{1}{1}$ ，选择出剪刀的概率为 0，选择出布的概率为 0。

双边匹配问题

稳定匹配，与算法导论相同

单边匹配算法

最大交易圈算法(TTC):

- 每个交易者连接一条指向他最喜欢的标的物的边，并从每一个标的物连接到其占有者或是具有高优先权的交易者，形成一张有向图
- 如果存在交易圈，其中的交易者，将每人指向的标的物赋予其，同时交易者放弃原先的标的物，相关占有者和标的物离开市场
- 从剩余的交易者和标的物之间重复交易圈匹配，直到没有交易圈

人工智能安全

- 加密协议
- 数字水印
- 数据安全与模型安全：鲁棒性、正确性、通用性
- 白盒攻击
- 生成对抗网络

九、雨课堂

本部分摘取了雨课堂上的部分题目。注意复习雨课堂的题。

题目：你有一个 $7 \times 7 \times 8$ 的输入，并使用 “pad = 2” 进行填充，填充后的尺寸是多少？

解析：pad=2指的是在图像的上下左右各加上2个单位的0，但是通道数不用增加。所以填充后的尺寸为 $11 \times 11 \times 8$

题目：假设你的输入是 100×100 彩色（RGB）图像，并且你使用卷积层和100个过滤器，每个过滤器都是 3×3 的大小，请问这个隐藏层有多少个参数（包括偏置参数）？

解析：由于输入的是彩色图像，因此通道数为3，过滤器的通道数也为3，所以每个过滤器有 $3 \times 3 \times 3 = 27$ 个参数，除此之外每个过滤器各有一个偏置参数，因此总参数个数为 $(27 + 1) \times 100 = 2800$ 个

题目：假设你的输入是一个 100×100 的彩色（RGB）图像，而你并没有使用卷积神经网络。如果第一个隐藏层有100个神经元，每个神经元与输入层进行全连接，那么这个隐藏层有多少个参数（包括偏置参数）？

解析：全连接，就是每个神经元需要与图像的每个输入都要进行连接，而彩色图像的通道数为3，一共可以看成 $100 \times 100 \times 3 = 30000$ 个输入，因此这里的参数个数为 $30000 \times 100 = 300,000$ ，由于每个神经元有一个偏置参数，所以总共参数个数为 $300000 + 100 = 300,100$ 个

你有一个 $16 \times 16 \times 8$ 的输入，并使用步幅为2、过滤器大小为2的最大化池，请问输出是多少？

解析：池化操作不改变通道数。步幅为2、过滤器大小为2的最大化池会将长宽减半，所以输出是 $8 \times 8 \times 8$

你有一个 $31 \times 31 \times 16$ 的输入，有16个过滤器进行卷积，每个过滤器的大小为 5×5 ，步幅为1，你想要使用“same”的卷积方式，请问pad的值是多少？

解析：pad的值是2

十、6.19更新

逻辑等价

命题逻辑

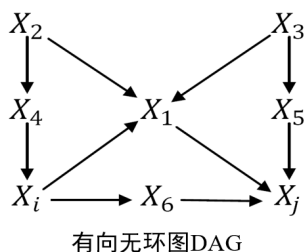
• 逻辑等价的例子

$\alpha \wedge \beta \equiv \beta \wedge \alpha$ ($\wedge$ 的交互律)	$(\alpha \rightarrow \beta) \equiv \neg \alpha \vee \beta$ (蕴涵消除)
$\alpha \vee \beta \equiv \beta \vee \alpha$ ($\vee$ 的交互律)	$(\alpha \leftrightarrow \beta) \equiv (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$ (双向消除)
$(\alpha \wedge \beta) \wedge \gamma \equiv \alpha \wedge (\beta \wedge \gamma)$ ($\wedge$ 的结合律)	$\neg(\alpha \wedge \beta) \equiv (\neg \alpha \vee \neg \beta)$ (De Morgan)
$(\alpha \vee \beta) \vee \gamma \equiv \alpha \vee (\beta \vee \gamma)$ ($\vee$ 的结合律)	$\neg(\alpha \vee \beta) \equiv (\neg \alpha \wedge \neg \beta)$ (De Morgan)
$\neg(\neg \alpha) \equiv \alpha$ (双重否定)	$(\alpha \wedge (\beta \vee \gamma)) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$ ($\wedge$ 对 $\vee$ 的分配律)
$(\alpha \rightarrow \beta) \equiv \neg \beta \rightarrow \neg \alpha$ (逆否命题)	$(\alpha \vee (\beta \wedge \gamma)) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$ ($\vee$ 对 $\wedge$ 的分配律)

因果推理

因果推理：有向无环图（DAG）

- 一个有向无环图唯一地决定了一个联合分布
- 一个联合分布不能唯一地决定有向无环图
 - 反过来的结论不成立
 - 如联合分布 $P(X_1, X_2) = P(x_1)P(x_2|x_1) = P(x_2)P(x_1|x_2)$



有向无环图DAG

联合分布可表示为：

$$\begin{aligned} &P(X_1, X_2, X_3, X_4, X_5, X_6, X_i, X_j) \\ &= P(X_2) \times P(X_3) \times P(X_1|X_2, X_3, X_i) \\ &\quad \times P(X_4|X_2) \times P(X_5|X_3) \times P(X_6|X_i) \times P(X_j|X_1, X_5, X_6) \end{aligned}$$

树搜索和图搜索区别

图搜索不允许访问重复的节点，即不允许访问的路径中有环。如果想添加搜索过的起点，可以选择保留代价最小的路径。

决策树

决策树的表和过程太长了，所以单独做成了一个pdf。

机器学习的分类、监督学习的分类

机器学习的分类：

- 监督学习：数据有标签，一般为回归或分类等任务。线性判别分析（LDA）也是监督学习。
- 无监督学习：数据无标签，一般为聚类或若干降维任务。
- 强化学习：序列数据决策学习，一般为从环境交互中学习。

监督学习的分类：

- 判别模型：
 - 判别方法直接学习判别函数 $f(X)$ 或者条件概率分布 $P(Y|X)$ 作为预测的模型，即判别模型。
 - 判别模型关心在给定输入数据下，预测该数据的输出是什么。
 - 典型判别模型包括回归模型、神经网络、支持向量机和Ada boosting等。
- 生成模型：
 - 生成模型从数据中学习联合概率分布 $P(X,Y)$ （通过似然概率 $P(X|Y)$ 和类概率 $P(Y)$ 的乘积来求取）
 - 典型方法为贝叶斯方法、隐马尔可夫链

注意看标题里的各种方法属于哪种学习。

两种降维方法的比较

	线性判别分析	主成分分析
是否需要样本标签	监督学习	无监督学习
降维方法	优化寻找特征向量 w	优化寻找特征向量 w
目标	类内方差小、类间距离大	寻找投影后数据之间方差最大的投影方向
维度	LDA降维后所得到维度是与数据样本的类别个数 K 有关	PCA对高维数据降维后的维数是与原始数据特征维度相关

GAN概念

GAN是一种生成区别模型。生成对抗网络由一个生成器（generator，简称G）和一个判别器（discriminator，简称D）组成。GAN的核心是通过生成器和判别器两个神经网络之间的竞争对抗，不断提升彼此水平以使得生成器所生成数据（人为伪造数据）与真实数据相似，使判别器无法区分真实数据和生成数据。

标准化

监督学习：结构风险最小化

- 结构风险最小化(structural risk minimization)
 - 为了防止过拟合，在经验风险上加上表示模型复杂度的正则化项(regularizer)或惩罚项(penalty term)：

$$\min_{f \in \Phi} \frac{1}{n} \sum_{i=1}^n Loss(y_i, f(x_i)) + \lambda J(f)$$

在最小化经验风险与降低模型复杂度之间寻找平衡

• 结构风险最小化(structural risk minimization)

$$\min_{f \in \Phi} \frac{1}{n} \sum_{i=1}^n \text{Loss}(y_i, f(x_i)) + \lambda J(f)$$

常用的正则项方法包括L1正则项和L2正则项：其中L1使权重稀疏，L2使权重平滑。一句话总结就是：L1会趋向于产生少量的特征，而其他的特征都是0，而L2会选择更多的特征，这些特征都会接近于0。

实际上，除了监督学习这里有标准化，在深度学习的损失函数里也会加入标准化项，都是为了缓解过拟合问题。

摘一道雨课堂题的答案。

4. 加入 L_2 标准化（normalization）后，对于包含参数 w 的线性回归损失函数的标准形式为：

$$L = (Y - Xw)^T(Y - Xw) + \lambda w^T w \quad \text{其中} \lambda > 0$$

(1) 假设 L_2 标准化项被误写为 $\lambda Y^T Y$ ，请解释为什么该项起不到标准化的作用。

(2) 在上述 L_2 标准化中，如果 λ 小于0，请解释为什么起不到标准化的作用。

(1)

该标准化项与参数 w 无关，该项对 w 的导数永远为0，对 w 的优化求解没有作用。

(2)

L_2 标准化项通过惩罚过大的参数 w 来避免过拟合， λ 小于0意味着该损失函数倾向于更大的 w ，从而激励过拟合，失去了标准化的作用。

梯度下降

批量梯度下降：每次迭代采用整个训练集的数据更新权重。

随机梯度下降：每次迭代从样本中随机抽取一组数据，用该组数据更新权重。